DAS Collecting Failed Tests Results

Reference : RA 434

Description

This Reference Architecture (RA 434) demonstrates a practical use of the **Analytics Management Platform (AMP)** to collect data from a running test program.

The use case describes the creation of a **Data Application Server (DAS)** that:

- Runs on an MST or IGXL tester
- Collects **parametric and functional test results**
- **Saves only failed test data** to a report file

The DAS is implemented in **C# (.NET 8.0)** and is designed to generate a formatted `.txt` report each time a **sublot** runs.

General Technical Info

- **Language:** C#
- **Framework:** .NET 8.0
- **IDE:** Visual Studio 2022
- **Platform:** MST or IGXL
- **File Lifecycle:**
  - File is opened at sublot start
  - File is closed at sublot end

Report Format

Parametric Test Example:

```
[Touchdown_001]
S:0|TNum:1235|TName:MyTest1|R:8|LLM:-5|HLM:5|RS:3|LLS:3|HLS:3
S:2|TNum:1235|TName:MyTest1|R:8|LLM:-5|HLM:5|RS:3|LLS:3|HLS:3
S:0|TNum:1236|TName:MyTest1|R:-12|LLM:-5|HLM:5|RS:3|LLS:3|HLS:3
```

Functional Test Example:

```
FUNCTIONAL|S:0|TNum:6543|TName:FTest1
FUNCTIONAL|S:2|TNum:6543|TName:FTest1
```

Format Details:

- **S:** Site
- **TNum:** Test Number
- **TName:** Test Name
- **R:** Result
- **LLM / HLM:** Low/High Limit Measured
- **RS / LLS / HLS:** Scaling factors

[!NOTE] The source code will be available soon in a dedicated Git repo.

User Guide

Start the DAS

```
FailedTests.exe
```

Load the IGXL Test Program

- Open: `FailedTestsTP.igxl`

Open the Data Collect Setup

- Navigate to: **Main Tab** > **Data Collect Group** > **DataCollect Setup**

Set the Sublot ID

- Go to the **Lot Definition** tab
- Click "More…"
- Set the **Sub Lot** value (5th from the bottom on the right column)
- Click OK

Execute the Test Program

- Enable **Datalog On**
- Click **Run** several times

Close the Datalog

- Re-open **DataCollect Setup**
- Go to **Summary Report** tab
- Click **"Get Final Summary / End Lot"**
- DAS window should show `SubLot End` and the path to the file

View the Results

- Go to `C:\temp`
- Open the generated `.txt` file
- Results are grouped by **Touchdown ID**, showing only **failing sites**

Screenshots

[!NOTE] Coming Soon

Demo Video

[!NOTE] Coming Soon

Summary

RA 434 shows how AMP and DAS can be used to filter and report only failed test data, reducing file size and focusing on valuable debugging content. This provides a solid foundation for integrating intelligent data filtering into your production test flow.

Advanced Code Overview

Reference : RA 434

This page provides a technical deep dive into the source code of **RA_FAARCH_434**, which demonstrates how a DAS (Data Application Server) is used to collect failed test results in an AMP environment.

Project Overview

- **Language:** C# (.NET 8.0)
- **Structure:** Modular, event-driven design
- **Executable:** `FailedTests.exe`

The application listens to AMP messages during test execution, collects results, and logs only the failed tests (parametric and functional) into a report.

Key Components

`Program.cs`

Entry point for the DAS. Initializes listeners and starts execution.

`AMP_DASExecution.cs`

Orchestrates the main DAS lifecycle:

- SubLot start/end logic
- Test cycle handling
- Data collection triggering

`AMP_Messages_Processor.cs`

Processes raw AMP messages and dispatches them to handlers:

- Handles site/test info
- Routes messages to appropriate data models

AMP Communication

`AMPEvents.cs`

Defines AMP-related events and constants.

`DASListener.cs`

Listens for incoming AMP messages and manages connection state.

`RequestProcessor.cs`

Processes inbound communication requests and ensures they follow expected structure.

Data Models

`AMPMessageModel.cs`

Object model for parsing and holding AMP message details.

`AMP_MESSAGE_LIST.cs`

Maintains a categorized list of parsed AMP messages.

`FailedFunctionalTestModel.cs` / `FailedParametricTestModel.cs`

Specific models used to store failed test results.

`Touchdowns.cs`

Keeps track of touchdown events and test site association.

`FailedTestModel.cs`

Base class for both parametric and functional test models.

File Output Logic

`FailedTestsDataFile.cs`

Handles creation and writing of output files:

- Report file lifecycle (start/end)
- Appending formatted test result strings
- Formatting logic for both parametric and functional records

---

Utilities

`Utils.cs`

Common helper methods for:

- String parsing
- Test site formatting
- Field validation

Browse the Code

The full source code is located in the `/source` folder of this RA package.

Example:

- AMP_DASExecution.cs
- DASListener.cs
- FailedTestsDataFile.cs

Download Source Package

- Download FailedTests.zip

View API Documentation

- View Generated Code Documentation

> **NOTE**
>
> Actual file paths should reflect your deployment.

Summary

This RA showcases a robust and modular implementation of an AMP-connected DAS designed to extract only failed test information in a structured and scalable way.

Explore the source to dive deeper into the message processing and data modeling logic powering this application.